

第6章 继承和派生类

继承是面向对象程序设计中最重要机制之一。它改变了过去传统程序中频频改写、重写程序和程序难以重复使用的缺点。C++语言中的继承机制可以扩充和完善原有程序以适应新的需求。这样,不仅可以节省程序开发的时间和资源,而且为未来的程序设计积累了资源。继承的使用可以大大提高软件开发的效率、增加程序的重用性和减少代码及数据的冗余度。

通过本章的学习,应了解继承性、基类、派生类、单重继承、多重继承和多级继承的基本概念,理解三种继承方式的特征和意义、基类与派生类的关系,掌握派生类的定义方法、派生类的构造函数和析构函数的特征和执行顺序,掌握派生类对象的组成成分;了解赋值兼容规则。

6.1 继承概述

类与类之间可以有多种关系。前面已经说明了 C++中类与类之间通过包含子对象实现的组合关系(简称为“有”关系,也称为“has a”关系)和通过友元函数实现的协同关系(简称为“友”关系,也称为“friend”关系)。本章将要学习的是类与类之间通过继承机制实现的父子关系(简称为“是”关系,也称为“is a”关系)。

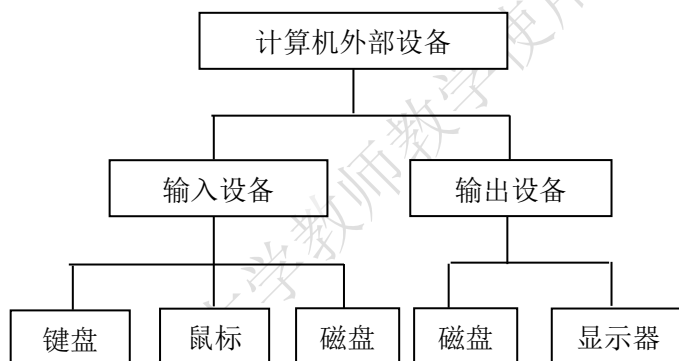


图 6.1 计算机外部设备组成图

所谓继承就是在原有类(称为基类或父类)的基础上,经过适当扩充和完善而建立新的类(称为派生类或子类)。这个新的类不仅具有基类的特征,而且还可以添加新的特征,同时能够继续被继承而派生出新的类。继承的本质是创建新类。而这个创建不是从头开始,而是在保留原有类的基础上建立起来的。

继承关系在现实世界的事物之间广泛存在着。人类、动物的继承关系自不必说。即使是计算机本身的继承关系也显而易见。图 6.1 是计算机外部设备组成图。它是一个分层图,好像一棵倒置的树一样,最高层是最普遍、最一般的,从上到下的每一层都比它的前一层更具体,但是较低层含有高层的信息,而又与高层有若干不同,所以,可以把上下层之间的关系看成基类和派生类之间的继承关系。例如,输入设备和输出设备都具备外部设备的特征,同时各自又具备新的特有性质,所以,输入设备和输出设备都是从外部设备派生出来的。同样,键盘和鼠标都是从输入设备派生出来的,显示器是从输出设备派生出来的。而磁盘则是从输入设备和输出设备两者派生出来的。因此,继承性是对客观世界固有现象的真实描述。

从一个派生类可以继承几个基类的角度来看,存在着三种基本继承关系。一个派生类可以从单个基类派生(称为单重继承),也可以从多个基类派生(称为多重继承),继承的基类中可以含有若干个派生类,即继承是多层次的(称为多级继承)。在实际应用中,也可能是三种基

本继承关系的组合(不妨称为组合继承)。

类之间的继承关系可以用有向无环图(DAG)表示,如图6.2所示。

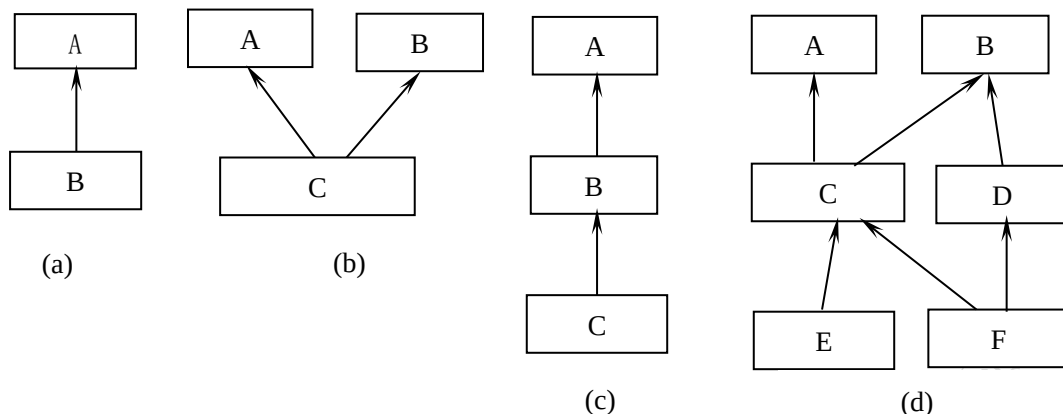


图 6.2 继承关系图

(a)B 单重继承 A;(b)C 多重继承 A 和 B;(c)多级继承;(d)混合继承

在多级继承关系中,与一个派生类直接相邻的基类称为该派生类的直接基类。反过来说,称该派生类为其直接基类的直接派生类。而与一个派生类不直接相邻的基类称为该派生类的间接基类,称该派生类为其间接基类的间接派生类。如图6.2(c)中的类A是类B的直接基类,是类C的间接基类。在树状继承关系图中处于最顶部的基类称为根基类,如图6.2(c)中的类A是根基类。

C++支持以上所述的各种继承关系。

6.2 基类和派生类

6.2.1 派生类的定义

派生类的定义格式为:

```
class 派生类名:继承方式 1 基类名 1,继承方式 2 基类名 2,...
```

派生类类体 ;

派生类类体中未定义的成员函数的实现

其中,基类名必须是已定义的类型名。当派生类名后面只带有一个基类名时,就是单重继承;当派生类名后带有多个基类名时,就是多重继承;当基类中包含已定义的派生类名时,就是多级继承。

继承方式有 **public**(公有继承)、**private**(私有继承,缺省继承方式)和 **protected**(保护继承)三种。无论何种继承方式,派生类中均包含基类的全体数据成员(静态数据成员除外)。

1. public 公有继承

在公有继承情况下,直接基类的公有成员和保护成员均分别作为派生类中的公有成员和保护成员。因此,在派生类的成员函数中,可以直接访问其直接基类的全体公有成员和保护成员。虽然派生类也继承了直接基类的私有成员,但不能直接访问直接基类的私有成员,只能通过直接基类的公有成员函数和保护成员函数间接访问它们。派生类的对象只能直接访问直接基类的公有成员。

例 6.1 公有继承示例。

```
class A {
public:
    int k1;    void f1();
protected:
```

```

        int j1;    void g1();
private:
        int i1;    void h1();           //不能被派生类直接访问
};
class B:public A {
public:
        int k2;    void f2();
protected:
        int j2;    void g2();
private:
        int i2;    void h2();
};

```

本例中,类 A 是类 B 的直接基类,类 B 是类 A 的直接派生类。类 B 公有继承类 A。这样,类 A 中的公有成员 `int k1` 和 `void f1()` 被继承作为类 B 的公有成员,类 A 中的保护成员 `int j1` 和 `void g1()` 被继承作为类 B 的保护成员。故在类 B 的成员函数 `f2()`、`g2()` 和 `h2()` 中,可直接访问类 A 的 `k1`、`f1()`、`j1` 和 `g1()`,但不能直接访问类 A 的 `i1` 和 `h1()`。

派生类 B 的对象仅可直接访问类 A 的 `k1` 和 `f1()`,而不能直接访问类 A 的任何保护成员和私有成员。

2. private 私有继承

在私有继承情况下,直接基类的公有成员和保护成员均作为派生类的私有成员。因此,在派生类的成员函数中,可以直接访问其直接基类的全体公有成员和保护成员。虽然派生类也继承了直接基类的私有成员,但不能直接访问直接基类的私有成员,只能通过直接基类的公有成员函数和保护成员函数间接访问它们。派生类的对象不能直接访问直接基类的任何成员。

例 6.2 私有继承示例。

```

class A {
public:
        int k1;    void f1();
protected:
        int j1;    void g1();
private:
        int i1;    void h1();           //不能被派生类直接访问
};
class B: A {           //缺省继承方式为 private
public:
        int k2;    void f2();
protected:
        int j2;    void g2();
private:
        int i2;    void h2();
};

```

本例中类 B 私有继承类 A。这样,类 A 中的公有成员 `int k1`、`void f1()` 以及保护成员 `int j1` 和 `void g1()` 均被继承作为类 B 的私有成员。故在类 B 的成员函数 `f2()`、`g2()` 和 `h2()` 中,可直接访问类 A 的 `k1`、`f1()`、`j1` 和 `g1()`,但不能直接访问类 A 的私有成员 `i1` 和 `h1()`。

派生类 B 的对象不能直接访问类 A 的任何成员。

3. protected 保护继承

在保护继承的情况下,直接基类的公有成员和保护成员均作为派生类的保护成员。因此,在派生类的成员函数中,可直接访问直接基类的全体公有成员和保护成员。虽然派生类也继承了直接基类的私有成员,但不能直接访问直接基类的私有成员,只能通过直接基类的公有成员函数和保护成员函数间接访问它们。派生类的对象不能直接访问直接基类的任何成员。

例 6.3 保护继承示例。

```
class A {
public:
    int k1;    void f1();
protected:
    int j1;    void g1();
private:
    int i1;    void h1();    //不能被派生类直接访问
};
class B: protected A {
public:
    int k2;    void f2();
protected:
    int j2;    void g2();
private:
    int i2;    void h2();
};
```

本例中类 B 保护继承类 A。这样,直接基类 A 中的公有成员 int k1、void f1()以及保护成员 int j1 和 void g1()均被继承为类 B 的保护成员。故在类 B 的成员函数 f2()、g2()和 h2()中,可直接访问类 A 的 k1、f1()、j1 和 void g1(),但不能直接访问类 A 的私有成员 i1 和 h1()。

派生类 B 的对象不能直接访问类 A 的任何成员。

从上面的例子可以看出,保护成员可以被继承,并且可以在派生类的成员函数中被直接访问,在这方面类似于公有成员;而保护成员不能被对象直接访问,在这方面它又类似于私有成员。

下面再看一个多级继承的例子。

例 6.4 多级继承示例。

```
class A {
public:
    int k1;    void f1();
protected:
    int j1;    void g1();
private:
    int i1;    void h1();    //不能被派生类直接访问
};
class B:public A {
public:
    int k2;    void f2();
protected:
```

```

        int j2;    void g2();
private:
        int i2;    void h2();
};
class C:public B {
public:
        void f3();
};
C c;

```

派生类 B 对其直接基类 A 的继承情况同例 6.1 中的说明。

派生类 C 对其直接基类 B 和间接基类 A 的继承情况为:把类 B 的公有成员 `int k2`、`void f2()`、类 A 的公有成员 `int k1` 和 `void f1()` 均继承为自己的公有成员,把类 B 的保护成员 `int j2`、`void g2()` 和类 A 的保护成员 `int j1`、`void g1()` 均继承为本类的保护成员。因此,类 C 的成员函数 `f3()` 可直接访问上述类 B 和类 A 的全体公有成员和保护成员。但不能直接访问类 B 和类 A 的任何私有成员。而类 C 的对象 `c` 只能直接访问类 A 和类 B 的公有成员,不能直接访问它们的任何保护成员和私有成员。

不同继承方式下基类与派生类的特性见表 6.1。在实际应用中,最常用的是公有继承。

表 6.1 不同继承方式下基类与派生类的特性

基类特性 派生类特性 继承方式			
	public	protected	private
public	public	protected	不可直接访问
protected	protected	protected	不可直接访问
private	private	private	不可直接访问

6.2.2 派生类对象的数据成员组成

无论基类与派生类属于何种继承关系,在派生类中只需定义自己的新成员,从而实现了原有类定义程序代码的重用,所以有人称类是“可重用的软件构件”。

必须明确,在继承情况下,派生类继承了直接和间接基类的除静态以外的全部数据成员。不管何种继承关系,也不管直接和间接基类数据成员具有何种访问权限,这一点都不会改变。

在这个前提下,一个派生类对象数据成员的组成情况可以用图形直观地表示出来,把这样的图称为派生类对象组成结构图。因为间接基类的数据成员包含在直接基类当中,同时考虑到类中可以包含若干个子对象,所以,一般的派生类对象组成结构图如图 6.3 所示。图中只给出直接基类的数据成员。在解决具体问题时,如果一个直接基类本身也是一个派生类,就需要把它的直接基类及子对象嵌入到组成结构图中。

如果把派生类对象组成结构图想像成一个无盖的桶,那么各直接基类的数据成员自桶底开始依次摆放,接着再依次摆放各子对象的数据成员,最上面摆放派生类自己的各数据成员。这就是说,各直接基类、各子对象及派生类自己的数据成员是“自底向上”依次摆放的。

派生类本身数据成员(除静态数据成员外)
子对象 m 数据成员(除静态数据成员外)
.....
子对象 2 数据成员(除静态数据成员外)
子对象 1 数据成员(除静态数据成员外)
直接基类 n 数据成员(除静态数据成员外)
.....
直接基类 2 数据成员(除静态数据成员外)
直接基类 1 数据成员(除静态数据成员外)

图 6.3 派生类对象组成结构图

例 6.5 画出下面程序中类 C 的对象 obj1 的组成结构图(图 6.4)。

```

class A {
public:
    int k1; void f1();
protected:
    static int j1;
    void g1();
private:
    int i1; void h1();
};
class B :public A{
public:
    void f2();
protected:
    int j2; void g2();
private:
    static int i2;
    A a1,a2;
    void h2();
};
class C :public B{
public:
    void f3();
protected:
    static int j3;
    void g3();
private:
    int i3;
};
C obj1;

```

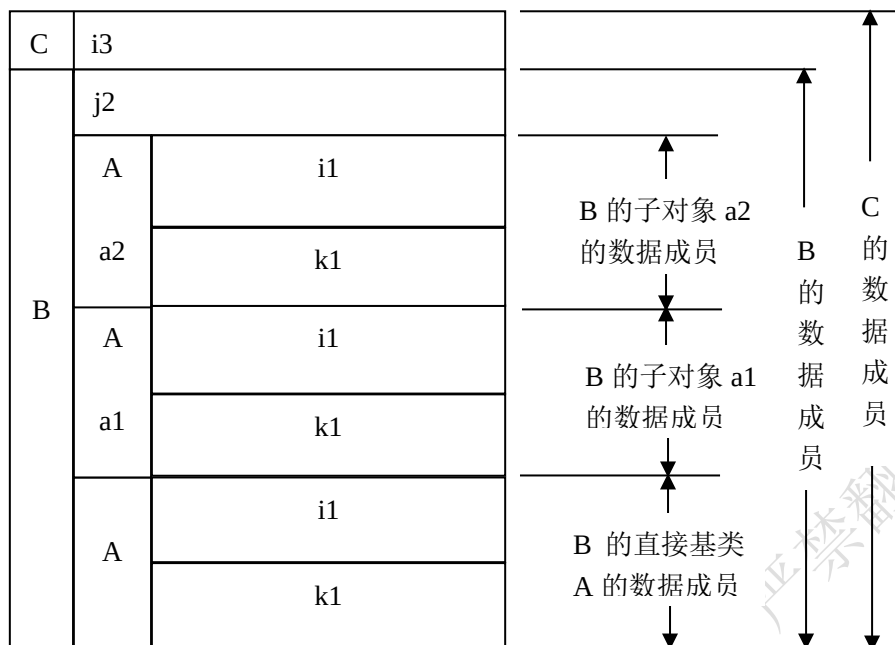


图 6.4 派生类 C 对象 obj1 的组成结构图

请读者牢记图 6.3 的派生类对象组成结构图，仔细体会、理解、记忆派生类对象中所含各种数据成员的种类和顺序，这对更好地理解、掌握派生类构造函数和析构函数等知识将有较大帮助。

6.3 派生类的构造函数和析构函数

基类的构造函数和析构函数均不能被派生类继承。这就是说，派生类对象中所含直接基类、子对象的数据成员，派生类本身是无法对它们直接进行初始化的。同时，派生类本身也无法直接释放派生类对象中所含直接基类和子对象的数据成员。因此，派生类的构造函数必须通过调用直接基类的构造函数对直接基类数据成员进行初始化，必须通过调用子对象所属的类的构造函数对它们的数据成员进行初始化。在删除派生类的对象时，也将调用直接基类和子对象所属类的析构函数释放派生类对象所包含的直接基类和子对象的数据成员。

在 C++ 中，为了实现对所含直接基类、子对象的数据成员进行初始化，派生类的构造函数具有与普通类的构造函数不同的形式。

6.3.1 派生类的构造函数

派生类构造函数的一般形式为：

派生类构造函数名(总参数表):直接基类初始化表,子对象初始化表,派生类自身数据成员初始化表

派生类构造函数体

其中:派生类构造函数名同本派生类名;总参数表包含其后各初始化表所需要的全体参数。

在上述形式中直接基类初始化表的形式为:

直接基类 1 构造函数 (参数表 1), 直接基类 2 构造函数 (参数表 2),...

子对象初始化表形式为:

子对象名 1(参数表 1),子对象名 2(参数表 2),...

派生类自身数据成员初始化表形式为:

数据成员名 1(参数 1),数据成员名 2(参数 2),...

当然,派生类自身数据成员初始化表也可以放到派生类构造函数体中,用赋值、输入或其

他形式对它们进行初始化。

从上述可以看出, 派生类构造函数的一般形式与普通类的构造函数的不同点是增加了直接基类初始化表。它们之间的共同点是都可能有子对象初始化表和派生类自身数据成员初始化表。

当创建派生类对象时, 系统自动调用派生类构造函数对派生类对象进行初始化。

在初始化过程中, 派生类构造函数的执行顺序是:

先执行所有直接基类构造函数(多重继承时, 按照定义派生类时各直接基类的出现顺序执行各直接基类构造函数), 然后, 再按定义子对象时各子对象的顺序执行各子对象相应类的构造函数, 最后执行对派生类自身数据成员的初始化和派生类自身的构造函数体。

对照图 6.3 的派生类对象组成结构图, 上面所说明的派生类构造函数的执行顺序恰好是“自底向上”进行各直接基类、各子对象和派生类自身数据成员的初始化。

值得注意的是, 当派生类构造函数中的直接基类初始化表为空时, 在创建派生类对象的过程中, 会自动按上述说明的顺序隐式地调用各直接基类的缺省构造函数, 对各直接基类的数据成员初始化; 当派生类构造函数中的子对象初始化表为空时, 在创建派生类对象的过程中, 会自动按上述说明的顺序隐式地调用子对象相应类的缺省构造函数, 对各子对象初始化。

关于派生类构造函数的调用时机, 可参看第 5 章普通类构造函数调用时机的说明和下面的例子。

例 6.6 单重继承派生类构造函数示例。

```
#include <iostream.h>
class Base {
public:
    Base(){ a=0;cout<<"Base 缺省构造函数\n"; }
    Base(int i){ a=i;cout<<"Base 带参构造函数\n"; }
    void print(){ cout<<"a="<<a<<" "; }
private:
    int a;
};
class Derived:public Base {
public:
    Derived(){ b=0;cout<<"Derived 缺省构造函数\n"; }
    Derived (int i,int k);
    void print();
private:
    int b;
};
Derived::Derived(int i,int k):Base(i),b(k) { cout<<"Derived 带参构造函数\n"; }
void Derived::print()
{ Base::print();          //调用基类 Base 的 print()函数
  cout<<"b="<<b<<endl;
}
void main()
{ Derived d1,d2(5,7);
  d1.print();   d2.print();
}
```


输出结果为:

```
Base 缺省构造函数
Derived 缺省构造函数
Base 带参构造函数
Derived 带参构造函数
a=0,b=0
a=5,b=7
```

本例中输出的前 2 行是创建派生类对象 d1 时先后在直接基类和派生类自身初始化过程中调用各缺省构造函数的输出结果。后续 2 行输出的是创建派生类对象 d2 时先后对直接基类和派生类自身初始化过程中调用各带参数构造函数的输出结果。

本例中派生类对象 d1、d2 的数据成员组成及其初始化后值的情况如图 6.5 所示。

本例中还有一条语句“Base::print();”比较特殊。由于派生类 Derived 与基类 Base 都含有 print() 函数, 所以, 为了表示在 Derived 中调用基类 Base 的同名函数 print(), 就使用在“print()”之前加上类属限定“Base::”方式。一般地讲, 要在派生类中访问基类的同名成员, 必须在该成员名前加上类属限定“基类名::”。如果不使用这个方法, 在派生类中将以派生类的成员覆盖基类的同名成员。假若去掉语句“Base::print();”中的“Base::”, 语法上没有问题, 编译程序一般不会给出错误或警告信息, 但是, 在程序执行期间, 当调用派生类 print() 函数时, 将会无限次地递归调用 print(), 以致造成内存资源的耗尽而“死机”。

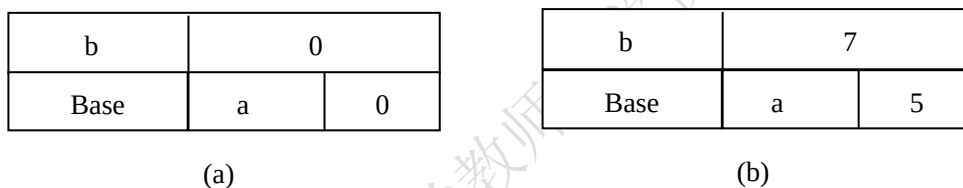


图 6.5 派生类对象 d1 和 d2 的组成结构图

(a)派生类对象 d1 组成结构图; (b)派生类对象 d2 组成结构图

例 6.7 带子对象的单重继承派生类构造函数示例。

```
#include <iostream.h>
class Base {
public:
    Base(){ a=0;cout<<"Base 缺省构造函数\n"; }
    Base(int i){ a=i;cout<<"Base 带参构造函数\n"; }
    void print(){ cout<<"a="<<a<<" "; }
    int Geta(){ return a; }
private:
    int a;
};
class Derived:public Base {
public:
    Derived(){ b=0;cout<<"Derived 缺省构造函数\n"; }
    Derived (int i,int j,int k);
    void print();
private:
```

```

    int b;
    Base Ba;                                //子对象或称为内嵌对象
};
Derived::Derived(int i,int j,int k):Base(i), Ba(j),b(k)
{ cout<<"Derived 带参构造函数\n"; }
void Derived::print()
{ Base::print();                          //调用基类 Base 的 print()函数
  cout<<"b="<<b<<"Ba.a="<<Ba.Geta()<<endl;
}
void main()
{ Derived d1,d2(5,3,7);
  d1.print(); d2.print();
}

```

输出结果为:

```

Base 缺省构造函数
Base 缺省构造函数
Derived 缺省构造函数
Base 带参构造函数
Base 带参构造函数
Derived 带参构造函数
a=0,b=0,Ba.a=0
a=5,b=7,Ba.a=3

```

本例中输出的前 3 行是创建派生类对象 **d1** 时先后在直接基类、子对象和派生类自身初始化过程中调用各缺省构造函数的输出结果。后续 3 行输出的是创建派生类对象 **d2** 时先后对直接基类、子对象和派生类自身初始化过程中调用各带参数构造函数的输出结果。

本例中派生类对象 **d1**、**d2** 的数据成员组成及其初始化后值的情况如图 6.6 所示。

b	0	
Base Ba	a	0
Base	a	0

(a)

b	7	
Base Ba	a	3
Base	a	5

(b)

图 6.6 派生类对象 **d1** 和 **d2** 的组成结构图

(a) 派生类对象 **d1** 组成结构图; (b) 派生类对象 **d2** 组成结构图

6.3.2 派生类的析构函数

派生类析构函数的形式无特殊之处。它不必显式地调用各直接基类、各子对象相应类的析构函数,这些操作是由系统自动完成的。

当删除一个派生类的对象时,所执行的各析构函数的顺序恰好与执行各构造函数的顺序相反,即先执行派生类的析构函数,再按各子对象创建的相反顺序执行各子对象相应类的析构函数,最后按各直接基类构造函数调用的相反顺序执行各直接基类的析构函数。

对照图 6.3 的派生类对象组成结构图,上面所说明的派生类析构函数的执行顺序恰好是“自顶向下”先后分别释放派生类自身、各子对象和各直接基类数据成员的。

例 6.8 单重继承派生类析构函数示例。

```
#include <iostream.h>
class Base{
public:
    Base(){ a=0; }
    Base(int i){ a=i; }
    void print(){ cout<<"a="<<a<<" "; }
    ~Base(){ cout<<"Base(a="<<a<<" )析构函数\n"; }
private:int a;
};
class Derived:public Base {
public:
    Derived(){ b=0; }
    Derived(int i,int k):Base(i){ b=k; }
    ~Derived(){ cout<<"Derived(b="<<b<<" )析构函数\n"; }
    void print();
private:
    int b;
};
void Derived::print() { Base::print(); cout<<"b="<<b<<endl; }
void main()
{ Derived d1,d2(5,3);
  d1.print(); d2.print();
}
```

输出结果为:

```
a=0,b=0
a=5,b=3
Derived(b=3)析构函数
Base(a=5)析构函数
Derived(b=0)析构函数
Base(a=0)析构函数
```

上述输出结果的后 4 行分别是删除派生类对象 d2 和 d1 时执行各析构函数的结果。

例 6.9 带子对象的单重继承派生类析构函数示例。

```
#include <iostream.h>
class Base{
public:
    Base(){ a=0; }
    Base(int i){ a=i; }
    void print(){ cout<<"a="<<a<<" "; }
    int Geta(){ return a; }
    ~Base(){ cout<<"Base(a="<<a<<" )析构函数\n"; }
private:int a;
```

```

};
class Derived:public Base {
public:
    Derived(){ b=0; }
    Derived(int i,int j,int k):Base(i),Ba(j){ b=k; }
    ~Derived(){ cout<<"Derived(b="<<b<<")析构造函数\n"; }
    void print();
private:
    int b;Base Ba;
};
void Derived::print()
{ Base::print(); cout<<"b="<<b<<","Ba.a="<<Ba.Geta()<<endl; }
void main()
{ Derived d1,d2(5,3,7);
  d1.print(); d2.print();
}

```

输出结果为:

```

a=0,b=0,Ba.a=0
a=5,b=7,Ba.a=3
Derived(b=7)析构造函数
Base(a=3)析构造函数
Base(a=5)析构造函数
Derived(b=0)析构造函数
Base(a=0)析构造函数
Base(a=0)析构造函数

```

上述输出结果的后 6 行分别是删除派生类对象 **d2** 和 **d1** 时执行各析构造函数的结果。

例 6.10 多重继承中的构造函数和析构造函数示例。

```

#include <iostream.h>
class base1{
public:
    base1(int i){ i1=i;cout<<"base1(i1="<<i1<<")构造函数\n"; }
    ~base1(){ cout<<"base1(i1="<<i1<<")析构造函数\n"; }
private:
    int i1;
};
class base2{
public:
    base2(int i){ i2=i;cout<<"base2(i2="<<i2<<")构造函数\n"; }
    ~base2(){ cout<<"base2(i2="<<i2<<")析构造函数\n"; }
private:
    int i2;
};
class derived:public base1,public base2 {
private:

```

```

    int i3;
public:
    derived(int j1,int j2,int j3) :base2(j2),base1(j1)
    { i3=j3; cout<<"derived 构造函数\n"; }
    ~derived(){ cout<<"derived 析构函数\n"; }
};
void main() { derived d(5,4,3); }

```

输出结果为:

```

base1(i1=5)构造函数
base2(i2=4)构造函数
derived 构造函数
derived 析构函数
base2(i2=4)析构函数
base1(i1=5)析构函数

```

请读者分析这个例题程序的执行过程。

本例演示了派生类析构函数的调用顺序。如果在类的构造函数等成员函数中不存在对主存、外部文件等计算机资源的动态使用,当设计应用程序时,一般是不需要写出析构函数的。

6.3.3 虚基类

通过前面的介绍已知,创建一个派生类对象要经过三个步骤:①吸收基类中除构造函数和析构函数之外的全体成员;②根据继承方式改变对基类成员的访问权限,覆盖基类中与派生类同名的数据成员和同名且参数类型与个数也相同的成员函数;③添加派生类自己的新成员。

在多级、多重继承的情况下,派生类对象可能具有通过不同继承路径得到两个或两个以上的同一间接基类的同名成员。这就造成了所谓的多级、多重继承中的“二义性”问题。为此,在 C++ 中引入“虚基类”,以避免这种二义性问题。虚拟继承具有非常复杂的机制,此处不作更详细的讨论。

6.4 赋值兼容规则

赋值兼容规则是指在公有继承的前提下,基类对象与公有派生类对象之间的自动类型转换和赋值的规则。总体上说,由于派生类中包含从基类继承的成员,所以可以将派生类对象赋值给基类对象,在需要基类对象时可以用派生类对象代替。

具体来说,赋值兼容规则主要讨论在公有继承时基类对象与派生类对象之间的关系和基类对象的指针与派生类对象的指针之间的关系。具体有以下几点。

①可以用派生类对象给基类对象赋值,反之则不行。例如,设有基类 `base` 和派生类 `derived`, 那么

```

base b;derived d;
b=d;           //正确, 用派生类对象给基类对象赋值
d=b;           //错误, 用基类对象给派生类对象赋值

```

用派生类对象给基类对象赋值,将把派生类对象中所包含的基类的数据成员传送给基类对象。

②可以用派生类对象初始化基类对象或基类的引用。例如:

```

derived d;
base b(d);      //正确, 用派生类对象初始化基类对象
base &rb=d;     //正确, 用派生类对象初始化基类的引用

```

用派生类对象初始化基类对象的效果与用派生类对象给基类对象赋值的效果一致;用派生类对象初始化基类的引用效果是使基类的引用成为派生类对象中所含基类的别名。正因为如此,如果函数的形参是基类对象或基类对象的引用,则对应的实参可以是其派生类对象。

例 6.11 基类对象与派生类对象之间的关系。

```
#include <iostream.h>
class A {
    int x;
public:
    A(int x1){ x=x1; }
    void print(){ cout<<"x="<<x<<" "; }
};
class B:public A {
    int y;
public:
    B(int x1,int y1):A(x1){ y=y1; }
    void print(){ A::print(); cout<<"y="<<y<<endl; }
};
void main() {
    B b1(10,20); b1.print();
    A a(b1); //用派生类对象初始化基类对象
    a.print(); cout<<endl;
    B b2(10,30); b2.print();
    A &ra=b2; //用派生类对象初始化基类的引用
    ra.print(); cout<<endl;
    B b3(10,40); b3.print();
    a=b3; //用派生类对象给基类对象赋值
    a.print(); cout<<endl;
}
```

输出结果为:

```
x=10 y=20
x=10
x=10 y=30
x=10
x=10 y=40
x=10
```

③可以把派生类对象的地址赋给指向其基类对象的指针变量,反之则不行。例如:

```
base b,*pb;derived d,*pd;
pb = &d; //正确
pd = &b; //错误
```

在把派生类对象的地址赋给指向其基类对象的指针变量时,实际上该指针变量是指向了派生类对象中所包含的基类的数据成员。

④可以把派生类对象的指针赋给基类对象的指针,反之则错。例如:

```
base *pb;derived d,*pd=&d;
pb=pd; //正确
```

```
pd=pb;           //错误
```

在把派生类对象的指针赋给基类对象的指针时,实际上是使基类对象的指针指向派生类对象中所包含的基类的数据成员。

注意:上述 4 条规则只对公有继承有效。如果用于私有继承或保护继承,则会出现语法错误。

还应该注意,在用基类对象的指针指向派生类对象时,只能直接访问派生类对象中从基类继承的公有成员。如果要用基类对象的指针访问派生类的公有成员,则必须进行强制类型转换后才行。例如:

```
derived d;base *pb=&d;
pb->print();           //正确, 直接访问从基类继承的公有成员
((derived*)pb)->print(); //正确, 强制类型转换后直接访问派生类的公有成员
```

例 6.12 基类对象的指针与派生类对象的指针之间的关系示例。

```
#include <iostream.h>
class A {
    int x;
public:
    A(int x1){ x=x1; }
    void print(){ cout<<"x="<<x<<" "; }
};
class B:public A {
    int y;
public:
    B(int x1,int y1):A(x1){ y=y1; }
    void print(){ A::print(); cout<<"y="<<y<<endl; }
};
void main() {
    B b1(10,20),b2(3,2),*pb=&b2;
    A *pa=&b1;           //用基类对象的指针指向派生类对象
    pa->print(); cout<<endl;
    pb=&b2;               //把派生类对象的指针赋值给基类对象的指针
    pa->print();           //基类对象的指针直接访问从基类继承的公有成员
    cout<<endl;
    ((B*)pa)->print();//基类对象的指针经过强制类型转换后访问派生类的成员
}
```

执行结果为:

```
x=10
x=3
x=3 y=2
```

6.5 例题

例 6.13 单重继承派生类析构函数的示例。

```
#include<iostream.h>
class A{
private:
```

```

    int a;
public:
    A(){ a=0;cout<<"A's default constructor called.\n"; }
    A(int i){ a=i;cout<<"A's constructor called.\n"; }
    void print(){ cout<<a<<" "; }
    ~A(){ cout<<"A's destructor called.\n"; }
    int Geta(){ return a; }
};
class B: public A{
public:
    B(){ b=0;cout<<"B's default constructor called.\n"; }
    B(int i,int j,int k):A(i),aa(j){ b=k; cout<<"B's constructor called.\n"; }
    void print();
    ~B(){ cout<<"B's destructor called.\n"; }
private:
    int b;
    A aa;                //子对象,或称为成员对象
};
void B::print(){
    A::print();
    cout<<b<<" ", "<<aa.Geta()<<endl;
}
void main(){
    B bb[2]={B(1,2,5),B(3,4,7)};
    for(int i=0;i<2;i++) bb[i].print();
}

```

输出结果(输出行后面的注释是为了便于理解而加入的)为:

A's constructor called.	为基类成员初始化(bb[0])
A's constructor called.	为子对象初始化(bb[0])
B's constructor called.	为派生类成员初始化(bb[0])
A's constructor called.	为基类成员初始化(bb[1])
A's constructor called.	为子对象初始化(bb[1])
B's constructor called.	为派生类成员初始化(bb[1])
1,5,2	
3,7,4	
B's destructor called.	释放 bb[1]派生类成员
A's destructor called.	释放 bb[1]子对象
A's destructor called.	释放 bb[1]基类成员
B's destructor called.	释放 bb[0]派生类成员
A's destructor called.	释放 bb[0]子对象
A's destructor called.	释放 bb[0]基类成员

例 6.14 有一个机动车类 **vehicle**、卧车类 **car** 和卡车类 **truck** 是 **vehicle** 类的派生类,每个类有各自相关的数据和输出的内容。程序如下:

```
#include <iostream.h>
```



```

class vehicle {                //定义机动车类 vehicle
public:
    vehicle(int w,double r) { wheels=w; range=r; }
    void show() { cout<<"车轮="<<wheels<<"个\t 行程="<<range<<"公里"; }
protected:
    int wheels;                //车轮个数
    double range;              //行程公里数
};
class car:public vehicle {     //定义卧车类 car
public:
    car(int w,double r,int p=4) :vehicle(w,r) { passengers=p;}
    void show()
    { cout<<"小车:"; vehicle::show();
      cout<<"\t 载人="<<passengers<<"位"<<endl;
    }
private:
    int passengers;            //乘客人数
};
class truck:public vehicle {   //定义卡车类 truck
public:
    truck(int w,double r,int p=2,double a=20) :vehicle(w,r)
    { passengers=p; weight=a; }
    void show()
    { cout<<"卡车:"; vehicle::show();
      cout<<"\t 载人="<<passengers<<"位";
      cout<<"\t 载重量="<<weight<<"吨"<<endl;
    }
private:
    int passengers;            //乘客人数
    double weight;             //载重量
};
void main()
{ car V1(4,100000.0,3),V2(4,120000.0);
  truck V3(8,10000.0,3,50.0);
  V1.show(); V2.show(); V3.show();
}

```

运行结果为:

小车:车轮=4 个 行程=100000 公里 载人=3 位

小车:车轮=4 个 行程=120000 公里 载人=4 位

卡车:车轮=8 个 行程=10000 公里 载人=3 位 载重量=50 吨

例 6.15 赋值兼容性规则示例。

```

#include<iostream.h>
class base{ public: void who(){cout<<"base"<<endl;} };
class derive1:public base

```

```
{ public: void who(){cout<<"derive1"<<endl;} };  
class derive2:public base  
{ public: void who(){cout<<"derive2"<<endl;} };  
void main()  
{ base obj1,*p; derive1 obj2; derive2 obj3;  
  p=&obj1; p->who();  
  p=&obj2; p->who();  
  p=&obj3; p->who();  
  ((derive1*)p)->who(); obj2.who() ;  
  ((derive2*)p)->who(); obj3.who();  
}
```

输出结果为:

```
base  
base  
base  
derive1  
derive1  
derive2  
derive2
```

练习 6

1.回答下列问题:

(1)什么是基类,什么是派生类,什么是直接基类,什么是间接基类?

(2)公有继承、保护继承和私有继承的主要区别是什么?

(3)什么是单继承,什么是多继承?

(4)在 C++ 中类与类之间有哪些关系,各自的作用是什么?

(5)有人用下面的话来形容类与类之间关系,你觉得合适吗?

“是”关系:你的也是我的,我的还是我的

“有”关系:我中有你

“友”关系:你的事就是我的事

2.选择题。

(1)以下叙述中不正确的是_____。

A)在公有继承中,基类的 **public** 和 **protected** 成员的访问属性在派生类中不变,而基类的 **private** 成员不可访问

B)对于保护继承,基类的 **public**、**protected** 和 **private** 成员在派生类中的访问属性同 A

C)对于私有继承,基类的 **public** 和 **protected** 成员都以 **private** 成员的身份出现在派生类中,而基类的 **private** 成员为不可访问成员

D)对于保护继承,基类的 **public** 和 **protected** 成员在类外通过派生类的对象都不能访问

(2)如果一个函数被说明为一个类的友元,则_____。

A)该类可以使用该函数的局部变量 B)该函数可以使用该类中成员函数的局部变量

C)该函数只能使用该类的数据成员 D)该函数可以使用该类的任意成员

(3)在派生类的构造函数的成员初始化列表中,不能包含_____。

A)基类的构造函数 B)派生类中子对象的初始化

C)基类的子对象的初始化 D)派生类中数据成员的初始化

(4)设类 B 是由类 A 继承的。当创建一个类 B 的对象时,构造函数的调用顺序是_____。

A)先调用类 A 的构造函数,然后调用类 B 的构造函数

B)先调用类 B 的构造函数,然后调用类 A 的构造函数

C)不调用类 A 的构造函数

D)视定义时类 A 和类 B 出现的位置

(5)如果某类是从另一个类继承来的,则析构函数的调用顺序是_____。

A)与类的继承关系无关 B)先调用基类的析构函数,再调用本类的析构函数

C)不调用基类的析构函数 D)先调用本类的析构函数,再调用基类的析构函数

3.阅读下列程序并写出输出结果:

(1)#include <iostream.h>

```
class Base {
public:
    void seti(int x){ i=x; }
    int geti(){ return i; }
protected:
    int i;
};
class Derived:Base{
```

```

public:
    Base::setj;
    void setj(int x){ j=x; }
    int getij(){ return (j--)*(++i); }
protected:
    int j;
};
int main()
{
    Derived obj;
    obj.seti(5);
    obj.setj(7);
    cout<<obj.getij()<<endl;
    return 1;
}
(2)#include<iostream.h>
class A{
public:
    A(int i,int j){ a=i;b=j; }
    void Move(int x,int y){ a+=x;b+=y; }
    void Show(){ cout<<"<<a<<","<<b<<"\n"; }
private:
    int a,b;
};
class B:public A {
public:
    B(int i,int j,int k,int l):A(i,j),x(k),y(l){ }
    void Show(){ cout<<x<<","<<y<<endl; }
    void fun(){ Move(3,5); }
    void f1(){ A::Show(); }
private:
    int x,y;
};
void main(){
    A e(1,2); e.Show();
    B d(3,4,5,6); d.fun();
    d.A::Show(); d.B::Show(); d.f1();
}

```

4. 程序填空题。

(1)下面程序中包含一个人员类 **person**(包括编号和姓名),由此派生出学生类 **student** 和教师类 **teacher**。类 **student** 包括学号、姓名、班级和成绩;类 **teacher** 包括编号、姓名、部门和职称。根据不同的对象显示不同的内容。

```

#include <iostream.h>
#include <string.h>
class person {                //人员类 person

```

```

    int no;
    char name[10];
public:
    person(int n,char *p) { no=n; strcpy(name,p); }
    void display(){cout<<no<<" "<<name<<endl;}
};
class student:public _____ { //学生类 student
    char depart[10];
    int degree;
public:
    student(int n,char *s,char *d,int c):person( _____ )
        { strcpy(depart,d); degree=c; }
    void display()
    { cout<<"学号和姓名:";person::display();
      cout<<"班级:"<<depart<<" 成绩:"<<degree<<endl;
    }
};
class teacher:public _____ { //教师类 teacher
    char depart[10], prof[10];
public:
    teacher(int n,char *s,char *d,char *p):person( _____ )
        { strcpy(depart,d); strcpy(prof,p); }
    void display()
    { cout<<"编号和姓名:"; person::display();
      cout<<"部门:"<<depart<<" 职称:"<<prof<<endl;
    }
};
void main()
{ student a1(025,"张三","电子技术班",85);
  a1.display();
  teacher t1(316,"刘强","计算机系","教授");
  t1.display();
}

```

(2)下面的程序中包括一个圆类 `circle`、一个桌子类 `table` 和一个圆桌类 `roundtable` 的定义。其中，类 `roundtable` 继承了类 `circle` 和类 `table`。该程序能输出一个圆桌的高度、桌面面积和颜色。

```

#include<iostream.h>
#include<string.h>
class circle {
    double radius;
public:
    circle(double r=0) { radius=r; }
    double getarea() { return radius*radius*3.14159; }
};

```

```

class table {
    double height;
public:
    table(double h=0) { height=h; }
    double getheight() { return height; }
};
class roundtable: public _____, public _____ {
    char *color;
public:
    roundtable(double h=0,double r=0,char *c=0):table(h),circle(r)
    { color=new _____ ;
      _____(color,c);
    }
    char *getcolor() { return color; }
};
void main()
{ roundtable rt(1.25,2.3,"橙黄色");
  cout<<"圆桌的高度:"<<rt. _____ ()<<"米, 桌面面积"<<rt. _____ ()
  <<"平方米, 圆桌的颜色"<<rt. _____ ()<<endl;
}

```

5. 设计一个点类、一个正方形类和一个立方体类之间的继承程序，每类包含计算面积、体积的成员函数，并编写主函数对它们进行测试。

6. 设计一个 **father** 类、一个 **mother** 类和一个 **child** 类。其中 **child** 类继承 **father** 类和 **mother** 类。**father** 类和 **mother** 类都包含姓和名两个数据成员，**child** 类仅包含名数据成员。要求一个 **child** 类的对象能够输出其父、其母和自己的姓名。